

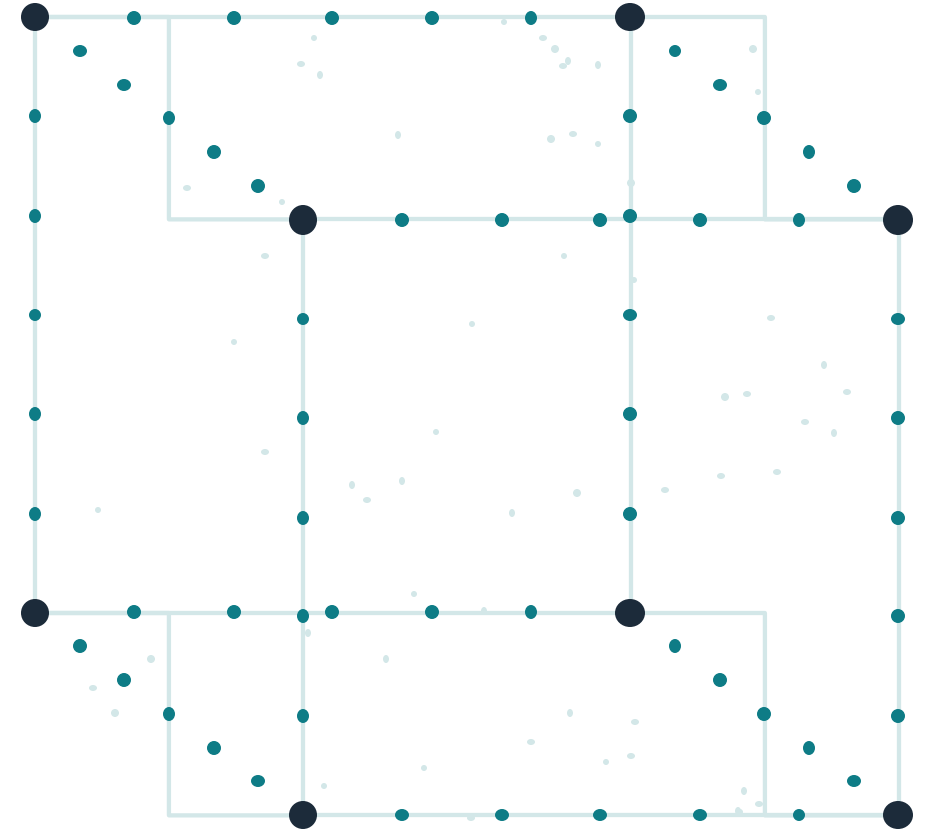
■ RRC SUMMER SCHOOL

3D Vision

Representations, Learning, and
Neural Scene Models

Akash Kumbar

Robotics Research Center



What we will cover today

01

Motivation

Why 3D vision is different from 2D, and why representation choice is everything.

02

Explicit representations

Point clouds, voxels, and meshes — storing geometry directly.

03

Datasets & annotations

From CAD objects to indoor scenes and self-driving — and how data shapes models.

04

Learning on 3D data

From voxel CNNs to PointNet and graph networks on raw points.

05

Implicit geometry

Occupancy Networks and DeepSDF — when a network becomes the shape.



01

Motivation

Why 3D vision is hard, and why the representation you pick decides everything that follows.

Why 3D vision?

- **Robots need geometry, not just appearance.** A flat image tells you what is there; 3D tells you where it is and what shape it has.
- **3D powers real tasks:** navigation, manipulation, mapping, AR/VR, autonomy, and reconstruction.
- **No single default format.** Unlike 2D images, 3D data comes in many competing representations.



Robotics & manipulation



Autonomous driving



AR / VR



3D reconstruction

Takeaway *The core question for this lecture: which representation should we use for the task?*

What makes 3D different from 2D?

Property	2D images	3D data
Structure	Regular pixel grid	Irregular sets, graphs, or fields
Sampling	Dense pixels	Sparse, range-dependent samples
Neighborhoods	Fixed and uniform	Non-uniform local density
Formats	Usually one: the image	Point cloud, voxel, mesh, implicit, splats
Annotation	Cheap at scale	Expensive and slow

Takeaway *Geometry, sensor noise, and representation choice are entangled — you cannot ignore any one of them.*

One scene, many representations

Point cloud

surface samples

Voxel grid

discretized space

Mesh

connected surface

Occupancy / SDF

continuous geometry function

NeRF

continuous radiance field

Gaussian splats

explicit radiance primitives

Takeaway *These are not just file formats — each makes different operations easy or hard.*

Representation depends on the task

Task	Common representation	Why it helps
Obstacle avoidance	occupancy grid / voxels	free vs occupied space is explicit
Semantic segmentation	point clouds	sensor-native per-point labels
Manipulation	point cloud / mesh / SDF	shape, contact, and distances matter
Simulation	mesh	surfaces, normals, collision geometry
Reconstruction	mesh / SDF / 3DGS	dense geometry or photorealistic rendering
Novel-view synthesis	NeRF / Gaussian splats	learn appearance from posed images



02

Explicit representations

Storing geometry directly — point clouds, voxels, and meshes, and the trade-offs each one forces.

Two families: explicit vs implicit

Explicit

Stores geometry directly

- Point cloud, voxel, mesh
- Easy to inspect and visualize
- Common in robotics pipelines
- Memory grows with detail

Implicit

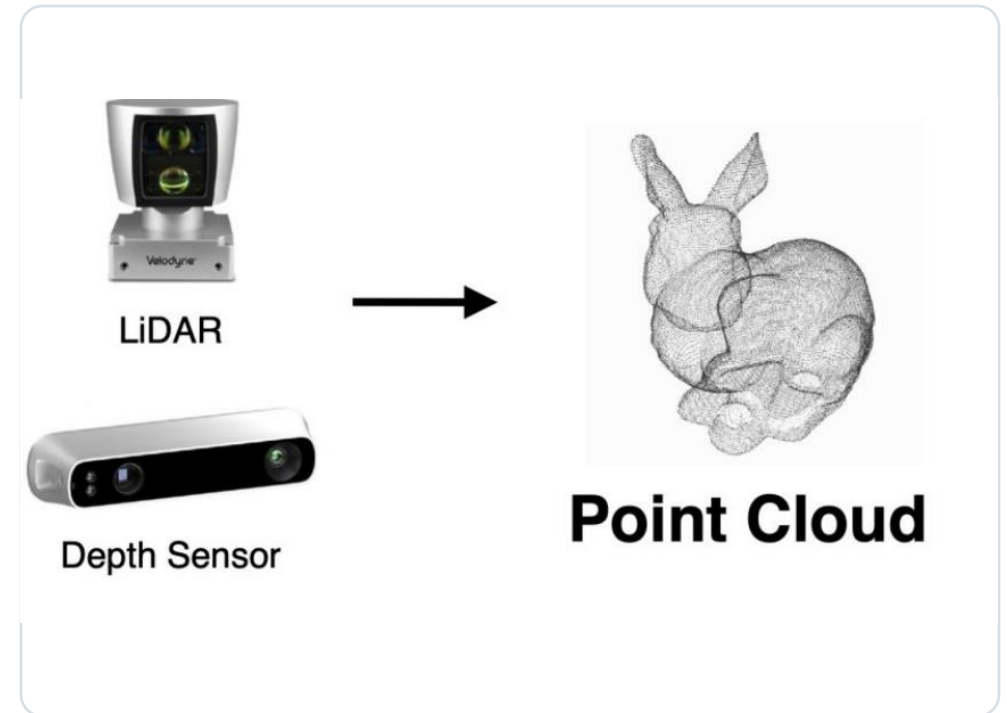
Stores geometry as a function

- Occupancy, SDF, NeRF
- Continuous queries at any point
- Common in neural reconstruction
- Needs decoding to a surface

Point clouds: sets of 3D samples

- **Where they come from:** LiDAR, depth cameras, stereo, or reconstruction.
- **What each point holds:** XYZ plus optional RGB, intensity, normal, or label.
- **Why robots like them:** they are the native output of most 3D sensors.
- **The catch:** no connectivity and no notion of free space.

$$P = \{ p_i \}, \quad p_i = (x_i, y_i, z_i, \text{features})$$



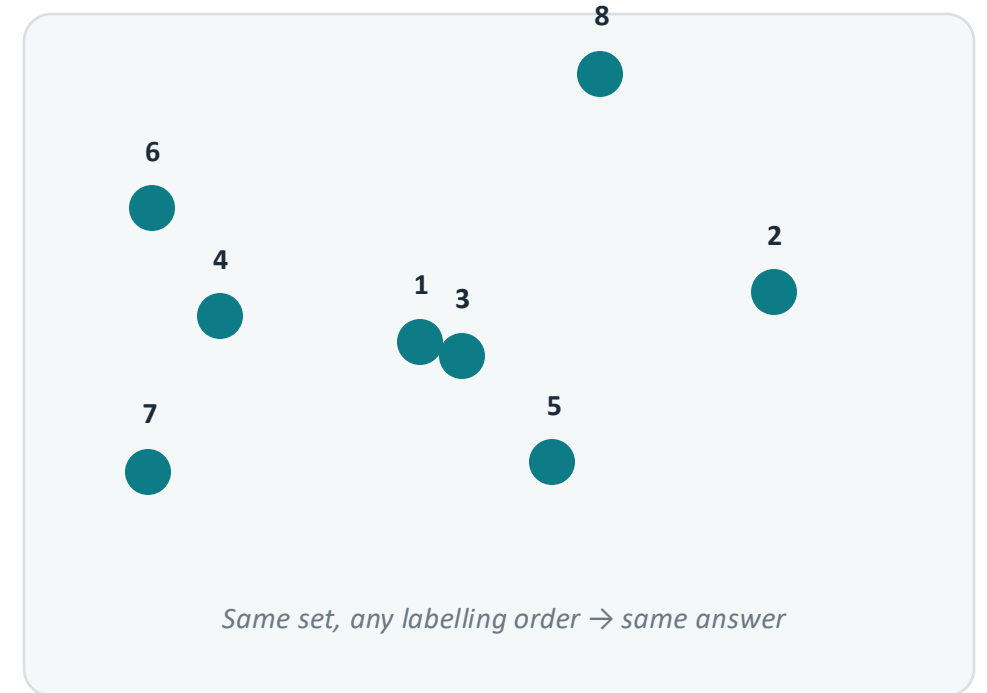
Sensors → a point cloud (Stanford bunny)

Why point clouds are challenging

- **Unordered:** no canonical order — the same cloud can be listed many ways.
- **Irregular:** point density varies across the scene.
- **Partial:** sensors only see visible surfaces.
- **Noisy & sparse:** returns depend on material, range, and viewpoint.

$$f(p_1, \dots, p_n) = f(p_{\pi(1)}, \dots, p_{\pi(n)})$$

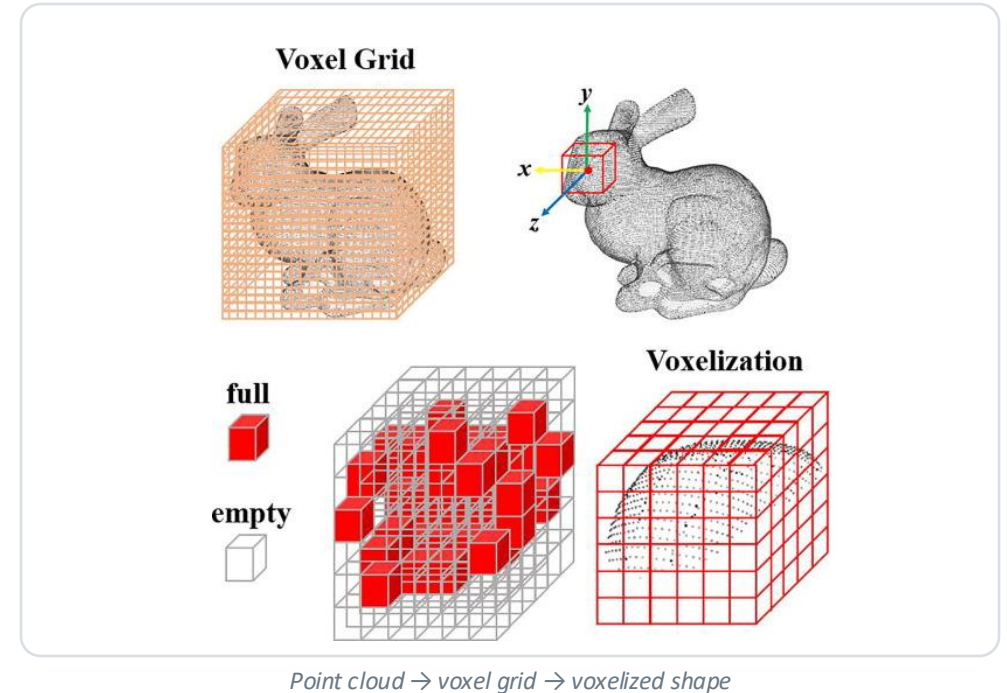
A good model must be permutation-invariant.



Voxels: turning 3D into a grid

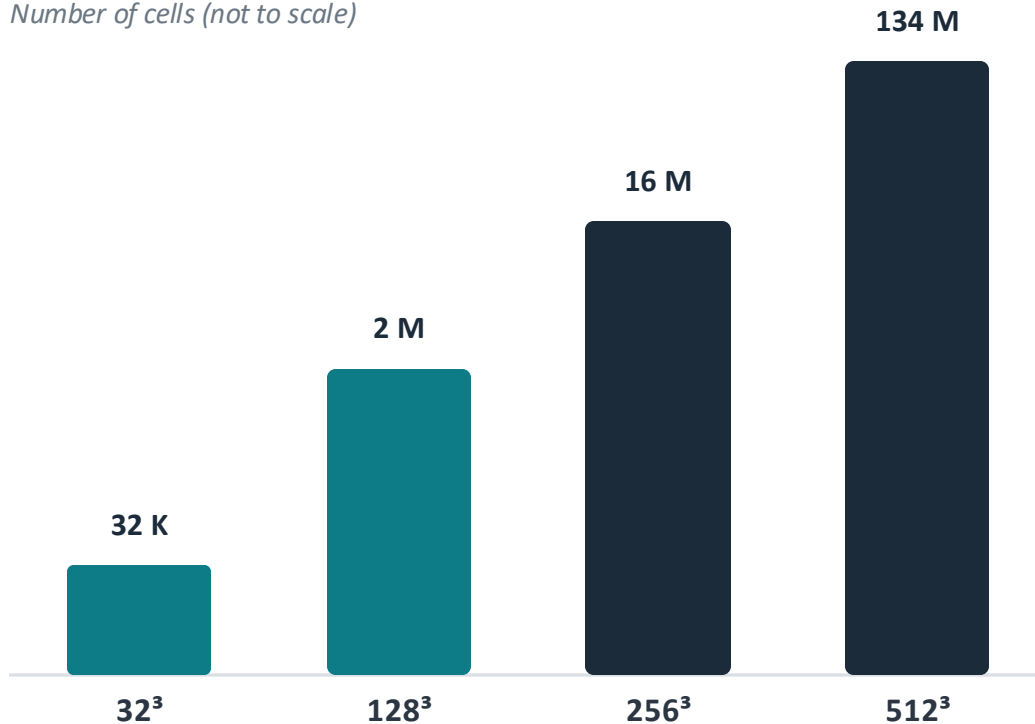
- **The idea:** chop 3D space into a regular grid of small cells.
- **Each cell can store:** occupancy, a TSDF/SDF value, a class, or features.
- **Why useful:** grids plug straight into 3D CNNs, mapping, and planning.
- **The price:** memory and resolution are tied together.

$$V \in \{0,1\}^{(H \times W \times D)}$$



Voxel resolution: detail vs memory

Number of cells (not to scale)



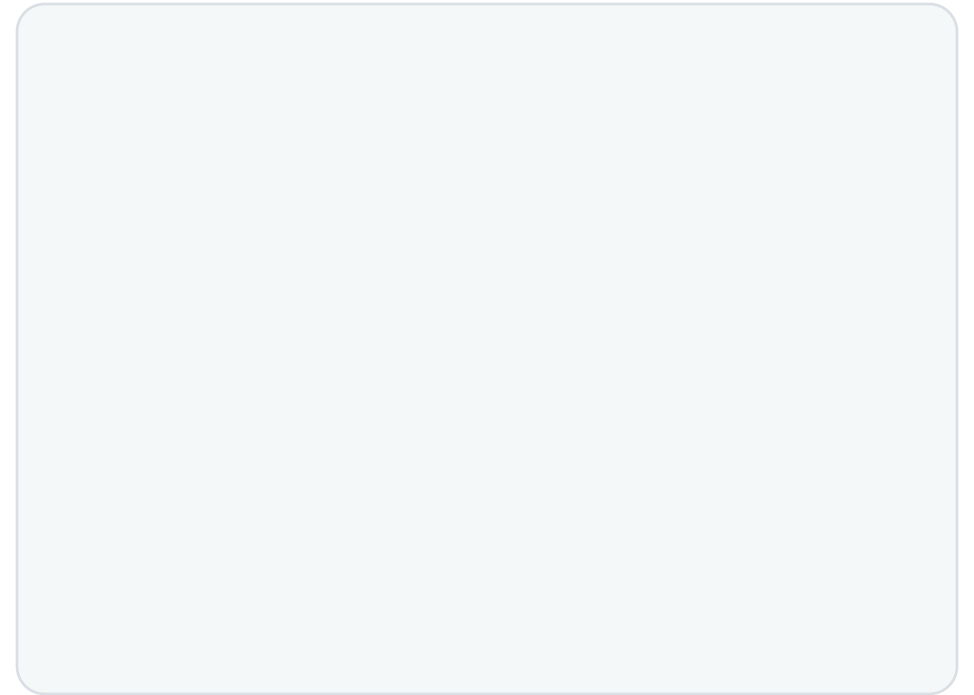
$$\text{memory} \propto R^3$$

- **Smaller voxels** preserve fine detail.
- **But cost grows cubically** — doubling resolution is 8× the memory.
- **Most cells are empty**, so dense grids waste huge amounts of space.

Meshes: connected surfaces

- **Vertices** define positions in space.
- **Edges** define which vertices are adjacent.
- **Faces** define the surface, usually triangles.
- **Great for** rendering, simulation, CAD, and collision geometry.
- **Harder for** generic neural nets — topology varies between shapes.

$$M = (V, E, F)$$



Vertices, edges, and triangular faces form the surface

Explicit representations at a glance

Representation	Stores	Strength	Weakness
Point cloud	surface samples	sensor-native, compact	unordered, no topology / free space
Voxel grid	discrete cells	CNN- and planning-friendly	cubic memory, quantization
Mesh	connected surface	rendering, simulation, collision	variable topology, holes
Multi-view	rendered images	reuses 2D CNNs	view selection, indirect 3D

Takeaway *No explicit representation wins everywhere — the right one depends on the operation you need.*



03

Datasets & annotations

What 3D data and labels actually look like — and why your dataset quietly decides your model.

Common 3D vision tasks

Task	Input	Output
Classification	object point cloud / mesh	one class label
Segmentation	scene or object points	a label per point
3D detection	LiDAR / RGB-D / camera	oriented 3D boxes
Pose / tracking	point cloud or RGB-D stream	object/camera pose over time
Reconstruction	images / depth / points	mesh, SDF, splats, or radiance field

Takeaway *Each task expects a different input and produces a different kind of geometric output.*

What does a 3D annotation look like?

● **Class label**
chair / car / mug

● **Point label**
wall / floor / table

● **Instance mask**
object ID per point

● **3D bounding box**
center + size + orientation

● **Camera pose**
intrinsics + extrinsics

● **Geometry target**
mesh, depth, occupancy, SDF

Takeaway *In 3D vision, the annotation format often determines which models you can even train.*

Object-level datasets: clean CAD shapes



ModelNet — CAD objects across 40 categories

Used for classification



ShapeNet — large repository of 3D models

ModelNet40

CAD objects for shape classification — clean, complete, object-centric.

ShapeNet

A large CAD repository used for classification and reconstruction.

- **Great for** learning the basics on controlled, complete shapes.
- **But** real robots see partial, noisy, cluttered scenes.

Part-level datasets: beyond a single label



PartNet — fine-grained, hierarchical part annotations

- **ShapeNetPart & PartNet** label the parts of an object, not just its class.
- **Annotations include** part labels, instance-level parts, and part hierarchies.
- **Why it matters:** parts often correspond to affordances and moving pieces.
- **Useful for manipulation** — handles, lids, drawers, buttons.

Indoor scene datasets: closer to real robots



ScanNet — reconstructed, labelled indoor scenes

ScanNet / ScanNet++

RGB-D scans with reconstructed geometry and dense labels.

S3DIS

Large indoor spaces with per-point semantic labels.

Matterport3D

Building-scale RGB-D with panoramas and meshes.

Takeaway *Indoor scenes add clutter, occlusion, and partial views — the messiness real robots face.*

Outdoor & autonomous-driving datasets

Multi-sensor datasets: calibrated LiDAR + cameras (sometimes radar) with vehicle poses.

KITTI

stereo, LiDAR, 3D boxes — the classic benchmark

SemanticKITTI

point-wise semantic labels on LiDAR scans

nuScenes

360° camera + LiDAR + radar, urban scenes

Waymo Open

large-scale LiDAR + camera with 3D boxes

Argoverse

HD maps with 3D tracking annotations

Lyft Level 5

multi-sensor fleet data for perception

Geiger et al., KITTI, CVPR 2012 · Behley et al., SemanticKITTI, ICCV 2019 · Caesar et al., nuScenes, CVPR 2020 · Sun et al., Waymo, CVPR 2020

Dataset choice shapes model choice

Dataset type	Typical data	Common task	Common models
CAD objects	meshes / sampled points	classification	VoxNet, PointNet
Indoor RGB-D	points / meshes / poses	segmentation	PointNet++, DGCNN, sparse convs
Autonomous driving	LiDAR + cameras	detection / segmentation	VoxelNet-style, sparse convs, fusion
Posed images	RGB + camera poses	novel-view synthesis	NeRF, Gaussian Splatting
Object scans	occupancy / SDF samples	reconstruction	Occupancy Nets, DeepSDF



04

Learning on 3D data

From voxel CNNs to PointNet and graph networks — how neural nets actually consume 3D.

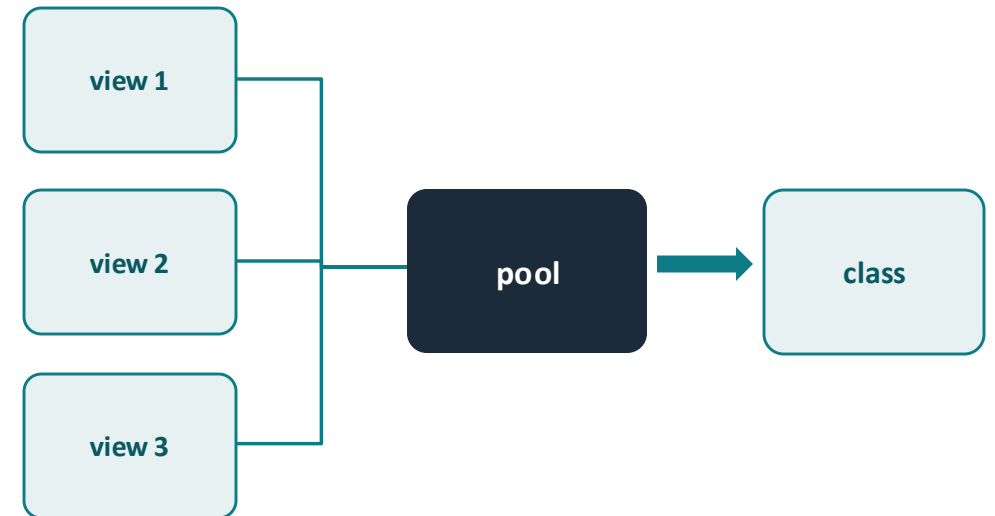
How do we feed 3D into a neural network?

Input strategy	Model idea	Example
Render views	reuse 2D CNNs	Multi-view CNN
Voxelize	apply 3D CNNs	VoxNet
Raw points	shared MLP + pooling	PointNet
Local neighborhoods	hierarchy / graph messaging	PointNet++, DGCNN
Modern encoders	sparse ops / attention / pretraining	Point Transformer v3

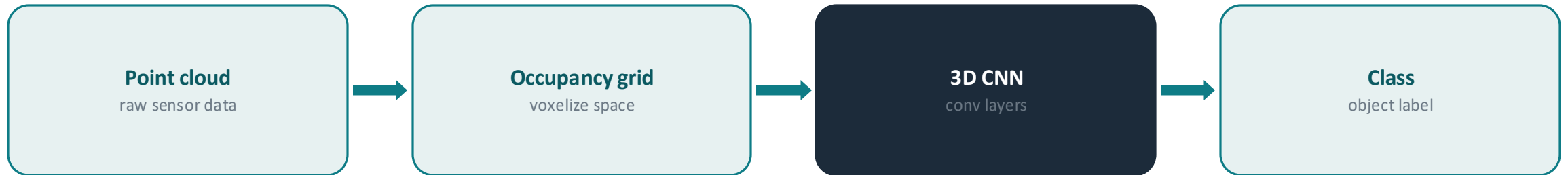
Takeaway *The history of 3D learning is really a search for the right way to handle irregular point sets.*

View-based learning: borrow 2D CNNs

- **Idea:** render an object from several viewpoints.
- **Run a 2D CNN** on each rendered image.
- **Pool the view features** into one descriptor.
- **Works well** when views are controlled (clean object scans).
- **Less natural** for raw robot point clouds and large scenes.



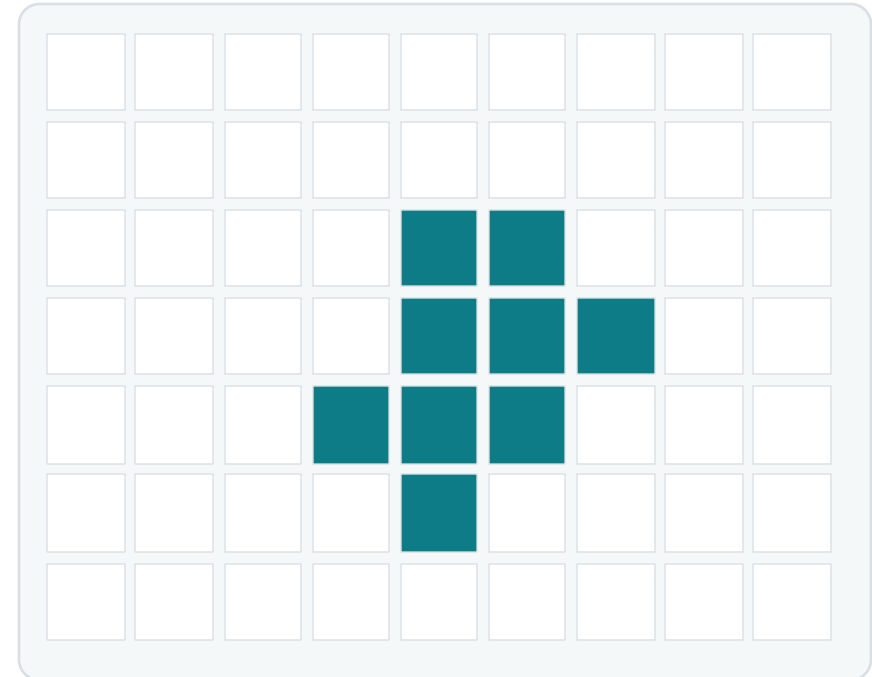
VoxNet: 3D CNNs on occupancy grids



- **An early, intuitive way** to bring CNNs to 3D — convolve over a voxel grid.
- **Strength:** reuses everything we know about convolutional networks.
- **Bottleneck:** grids are mostly empty and memory grows as R^3 .
- **This pressure** is exactly what pushes us toward point-based methods.

Why move beyond voxels?

- **Wasted memory:** most cells in a grid are empty.
- **Coupled cost:** resolution is tied to memory and compute.
- **Quantization:** snapping points to a grid loses information.
- **Scenes are big:** high resolution needs sparse or point-based methods.



Only the teal cells hold the surface — the rest is wasted

Takeaway *Can we learn directly on the points, with no grid at all?*

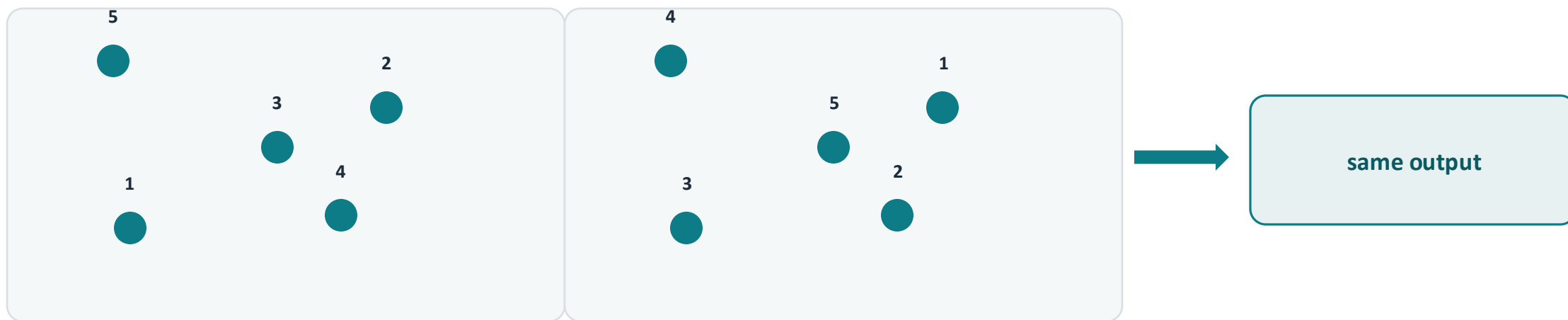
Learning directly on point clouds

- A **point cloud** is a **set**, not an image and not a sequence.
- The **order of points** in memory carries no meaning.
- So the **network** must give the same answer for any ordering.
- **Central design question:** how do we build permutation-invariant networks?

“PointNet provides a simple and elegant answer to this question.”

Permutation invariance is the key idea

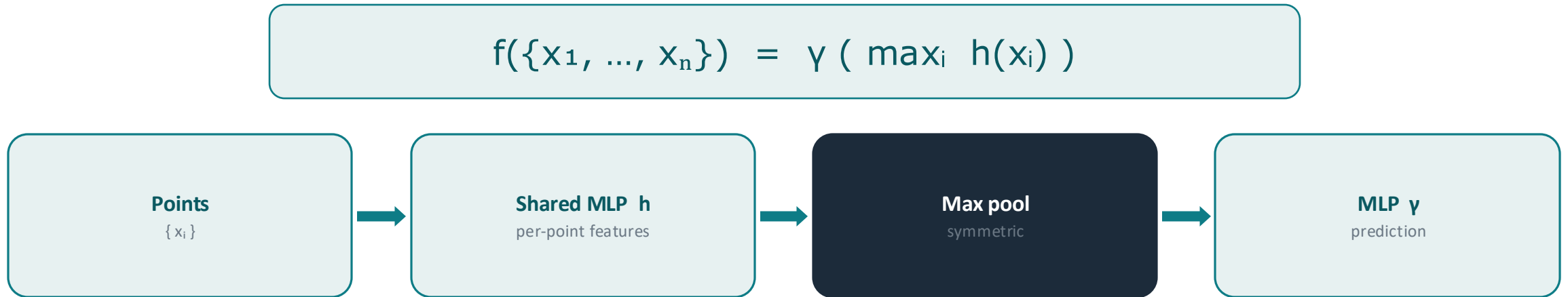
$$f(p_1, \dots, p_n) = f(p_{\pi(1)}, \dots, p_{\pi(n)})$$



Two orderings of the same set — the model must not care.

Takeaway *The max-pooling layer is not a detail; it is the trick that makes PointNet order-independent.*

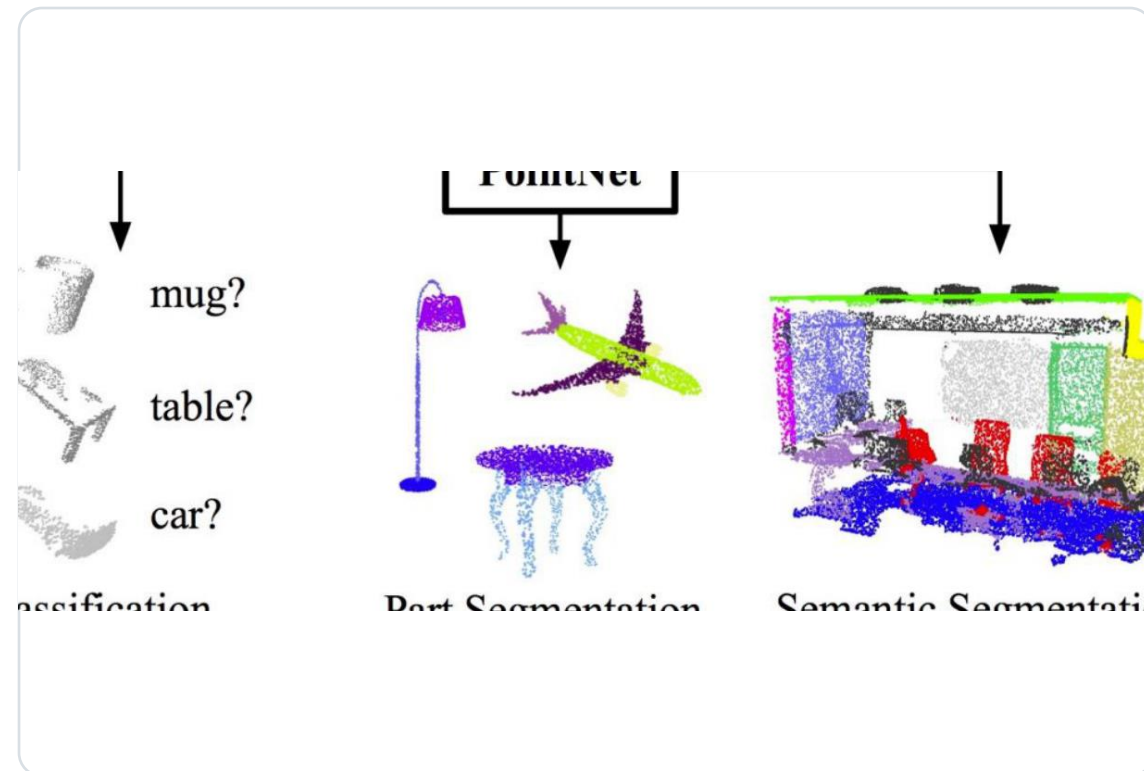
PointNet in one equation



- **h** — a shared MLP applied independently to every point.
- **max** — a symmetric pooling that collapses the set into one global feature.
- **γ** — an MLP mapping that global feature to a prediction.

One representation, many tasks

- **Classification** uses the global feature to predict one label.
- **Part segmentation** predicts a label for every point of an object.
- **Semantic segmentation** predicts a class for every point in a scene.
- **Same core idea**, different output heads on top.



PointNet: classification, part and semantic segmentation

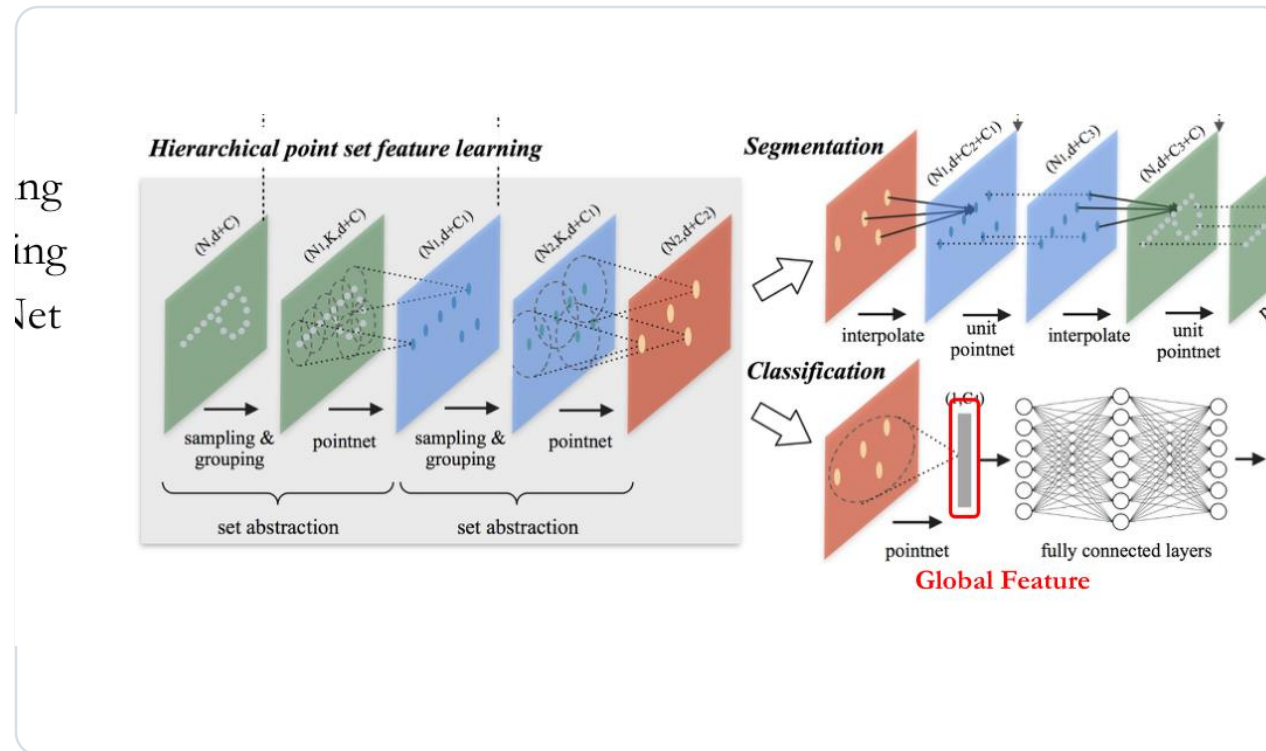
Where PointNet falls short

- **Points are processed** mostly independently before pooling.
- **Weak local geometry:** interactions between nearby points are limited.
- **Global feature** can depend heavily on absolute coordinates.
- **The fix:** add locality — neighborhoods, hierarchy, graphs, sparse convs, attention.

Next step

Give the network a sense of local neighborhoods, so it can read fine geometric structure.

PointNet++: add locality and hierarchy



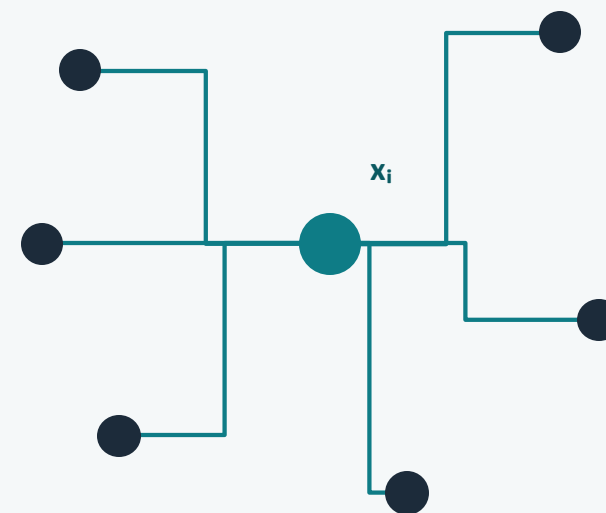
Hierarchical set abstraction: sample $\rightarrow$ group $\rightarrow$ local PointNet

- **Sampling** selects representative center points.
- **Grouping** builds a local neighborhood around each center.
- A **local PointNet** extracts features from each neighborhood.
- **Repeat** to grow features from local detail to global shape.

DGCNN / EdgeConv: let neighbors talk

- **Treat points as graph nodes** and neighbors as edges.
- **For each edge**, compute a feature from a point and its neighbor.
- **Aggregate** neighbor features with a symmetric function.
- **Dynamic graph**: neighborhoods can be recomputed at each layer.

$$e_{ij} = h_{\theta}(x_i, x_j - x_i)$$



Edges carry relative geometry $x_j - x_i$

The evolution of point-cloud encoders

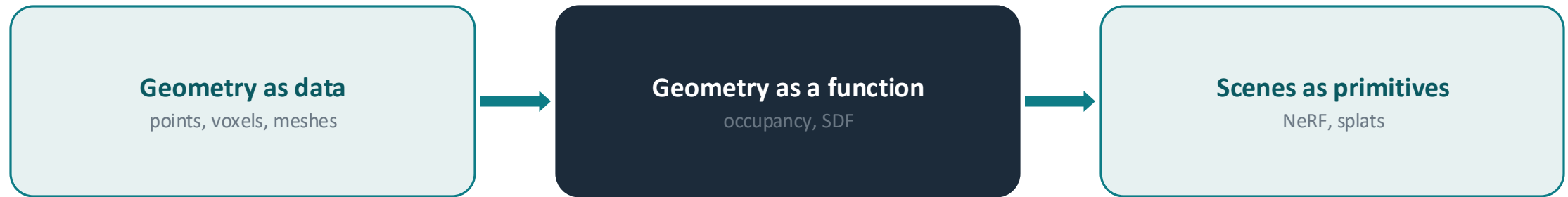
Model	Core idea
VoxNet	3D CNN on a voxel grid
PointNet	a learned set function over points
PointNet++ / DGCNN	local neighborhoods and graphs
Sparse convolutions	scale CNNs to large scenes
Point Transformer / PTv3	attention-based context
Foundation 3D encoders	one pretrained encoder across domains

The trend

From single-dataset, hand-built models toward scalable, pretrained 3D encoders — echoing the 2D vision story.

Qi 2017 · Wang 2019 · Choy et al. MinkowskiNet 2019 · Wu et al. PTv3, CVPR 2024

From understanding 3D data to representing 3D worlds



- **So far** we chose explicit representations, looked at datasets, and trained networks to classify, segment, and detect.
- **Next** we ask: can a neural network itself store the geometry?
- **That leads to** Occupancy Networks and DeepSDF — and later, radiance fields.



05

Implicit geometry

When the network stops processing 3D data and becomes the 3D representation itself.

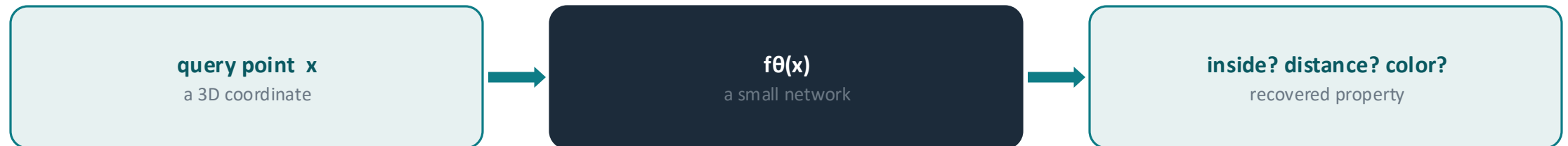
From stored geometry to queried geometry

Explicit

Store the object directly as points, voxels, or mesh elements. The data is the shape.

Implicit

Store a function. Query any 3D coordinate and recover occupancy, distance, density, or color.

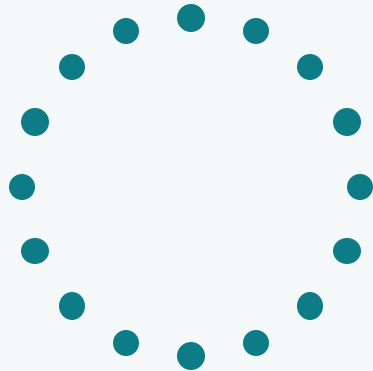


Takeaway Core idea: geometry is recovered as a level set of a learned function.

Implicit functions: a 2D intuition

Parametric form

Sweep ϑ to generate points on the curve.



$$x = r \cos\theta, \quad y = r \sin\theta$$

Implicit form

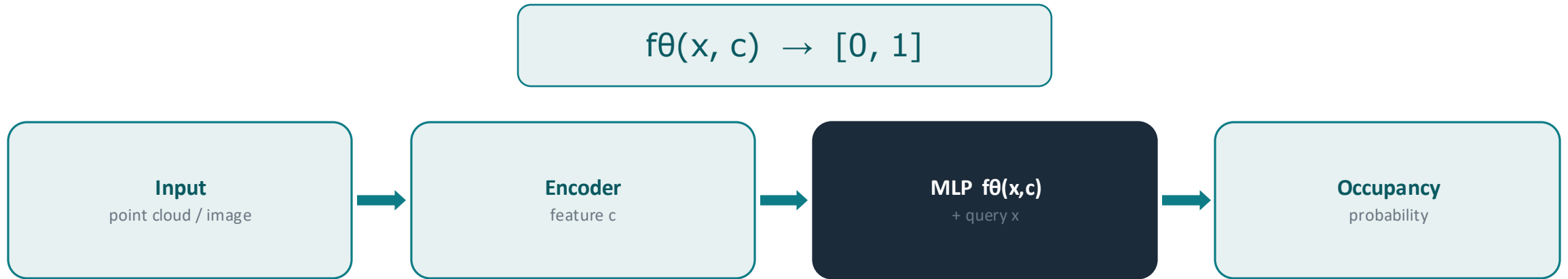
Answer a query: on, inside, or outside?



$$f(x,y) = x^2 + y^2 - r^2$$

Takeaway *Parametric form generates points; implicit form answers point queries.*

Occupancy Networks: inside-or-outside queries



- **Input**: a query point x in 3D space.
- **Condition c** : a feature from an image, point cloud, or voxel encoder.
- **Output**: the probability that x is inside / occupied.
- **The surface** is the decision boundary where the probability crosses τ .

$$\text{surface} \approx \{ x \mid f_{\theta}(x, c) = \tau \}$$

How occupancy learning works

1

Sample query points

Draw points near the surface and in the surrounding volume.

2

Label occupancy

Each sample gets $y = 1$ if inside, else $y = 0$.

3

Predict with an MLP

Condition on an encoder feature when reconstructing.

4

Extract the surface

Evaluate on a grid, then run Marching Cubes.

Takeaway *Training is just binary classification: cross-entropy between predicted and true inside/outside labels.*

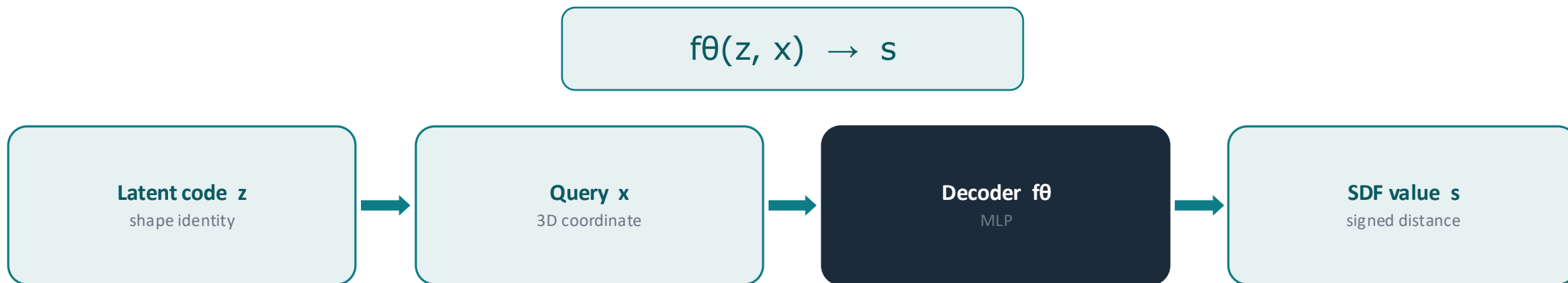
Lorensen & Cline, Marching Cubes, SIGGRAPH 1987

Occupancy field vs occupancy grid

	Occupancy grid	Occupancy field (network)
Storage	explicit cells in memory	weights of a learned function
Resolution	fixed by grid size	continuous — query anywhere
Memory	grows as R^3	roughly constant network size
Surface	read cell values	level set, extracted on demand

Takeaway *Don't think “better voxels” — occupancy networks move storage from grid cells to a query function.*

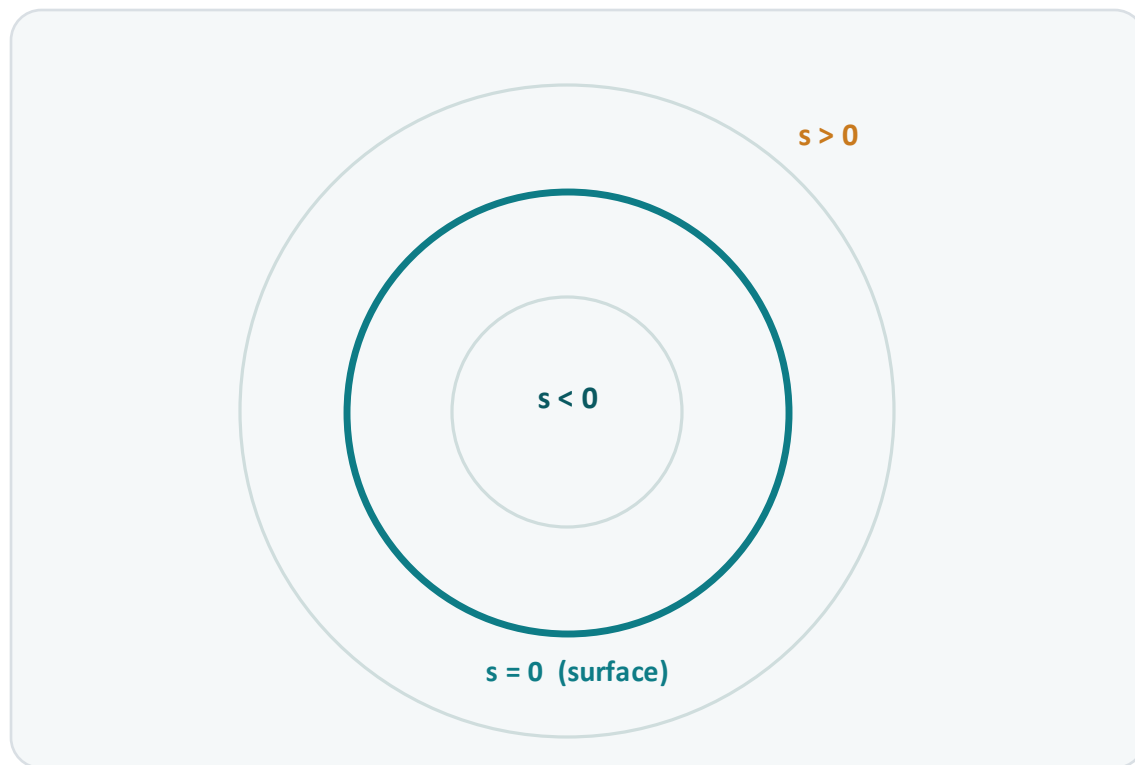
DeepSDF: learn a signed distance field



- **Predicts signed distance**, not just an inside/outside probability.
- **Magnitude** = distance to the closest surface.
- **Sign** = which side of the surface the point is on.
- **The shape** is the zero-level set of the field.

$$\text{surface} = \{ x \mid f_{\theta}(z, x) = 0 \}$$

SDF intuition: distance plus sign



- **The zero-level set** is the surface itself.
- **Distance magnitude** tells how far a query is from the surface.
- **The sign** tells inside (–) versus outside (+).
- **Gradients of the SDF** give surface normals for free.

Takeaway For robotics, distance queries connect naturally to collision checks and optimization.

Park et al., DeepSDF, CVPR 2019

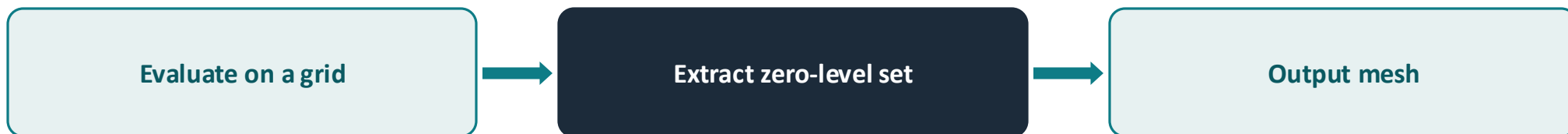
DeepSDF: training and inference

Training

Jointly learn the decoder weights θ and one latent code z per training shape, supervised by sampled 3D points and their SDF values.

Inference

For a new observation, optimize or predict a latent code z , then query $f_{\theta}(z, x)$ across space to reconstruct the shape.



Park et al., DeepSDF, CVPR 2019 · surface via Marching Cubes

Occupancy Networks vs DeepSDF

	Occupancy Networks	DeepSDF
Network predicts	inside/outside probability	signed distance value
Surface is	probability level set ($= \tau$)	zero-level set ($= 0$)
Extra signal	none beyond occupancy	distance + sign + normals
Many shapes	condition on encoder feature c	one decoder + per-shape code z
Mesh via	Marching Cubes	Marching Cubes

Takeaway *Both are implicit: they answer geometry queries instead of storing an explicit surface.*

Why implicit geometry matters for robotics

Useful for

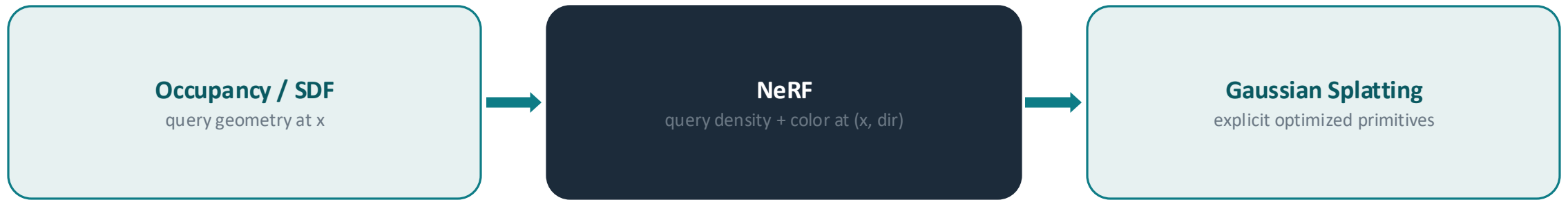
- Shape completion from partial scans
- Object reconstruction and manipulation
- Collision-aware optimization
- Continuous geometry queries

Still hard

- Surface extraction has a cost
- Generalization beyond training shapes
- Scaling to large, dynamic scenes
- Integration with planning maps

Takeaway *Object-centric? implicit fields shine. Large robot maps? explicit TSDF/occupancy is often still simpler.*

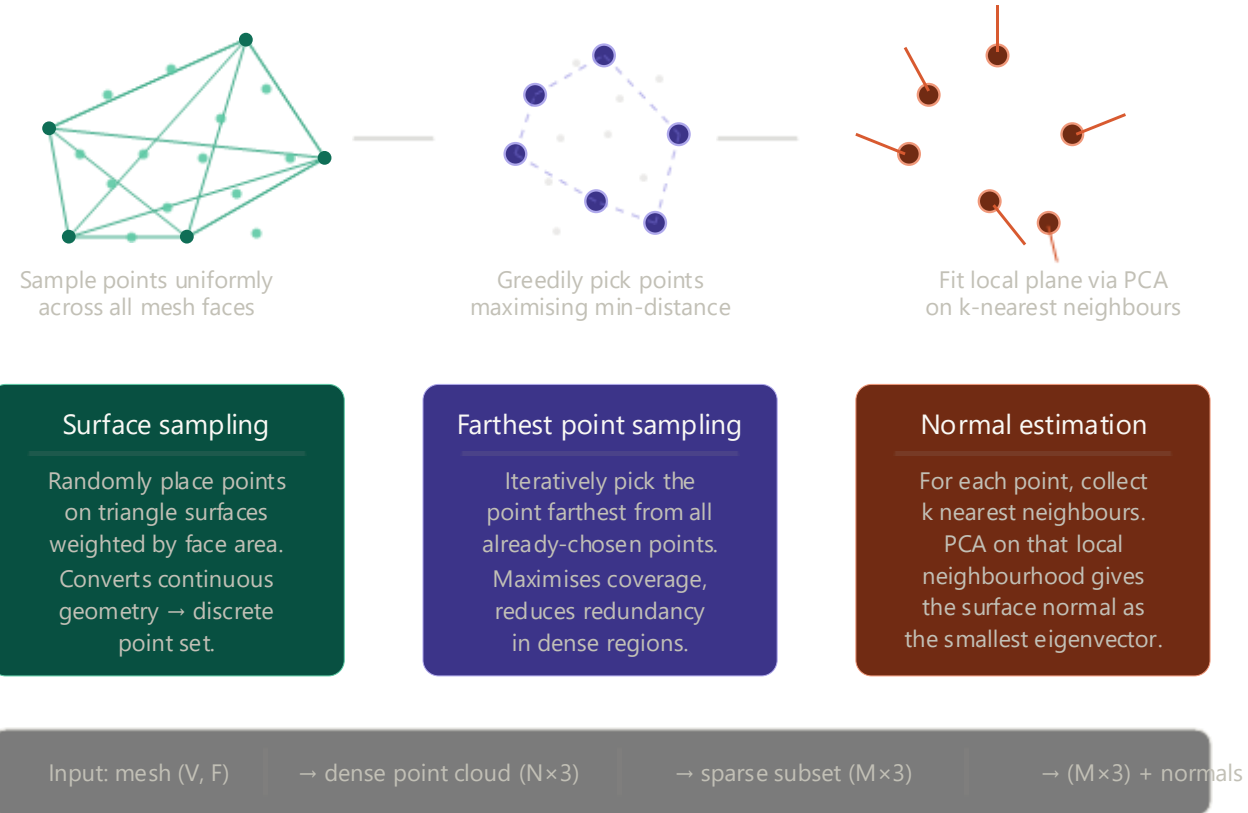
Next: from implicit geometry to neural rendering



The bridge: DeepSDF and Occupancy Networks teach the idea of continuous learned 3D functions. NeRF and 3D Gaussian Splatting apply the same idea to photorealistic scene reconstruction.

Mesh → point preprocessing (reference)

How ModelNet-style pipelines turn CAD meshes into point clouds for training.



Takeaway *These steps connect the mesh datasets of Part 3 to the point-based models of Part 4.*

References — models & methods

Su et al.. *Multi-view CNNs for 3D Shape Recognition*. **ICCV 2015**

Maturana & Scherer. *VoxNet: A 3D CNN for Real-Time Object Recognition*. **IROS 2015**

Qi et al.. *PointNet: Deep Learning on Point Sets*. **CVPR 2017**

Qi et al.. *PointNet++: Deep Hierarchical Feature Learning*. **NeurIPS 2017**

Wang et al.. *Dynamic Graph CNN (EdgeConv)*. **ACM TOG 2019**

Choy et al.. *4D Spatio-Temporal ConvNets (MinkowskiNet)*. **CVPR 2019**

Wu et al.. *Point Transformer V3*. **CVPR 2024**

Mescheder et al.. *Occupancy Networks*. **CVPR 2019**

Park et al.. *DeepSDF: Continuous Signed Distance Functions*. **CVPR 2019**

Lorensen & Cline. *Marching Cubes*. **SIGGRAPH 1987**

Mildenhall et al.. *NeRF: Neural Radiance Fields*. **ECCV 2020**

Kerbl et al.. *3D Gaussian Splatting*. **SIGGRAPH 2023**

References — datasets

Wu et al.. *3D ShapeNets / ModelNet*. **CVPR 2015**

Chang et al.. *ShapeNet: An Information-Rich 3D Model Repository*. **arXiv 2015**

Yi et al.. *ShapeNetPart (Scalable Active Framework)*. **SIGGRAPH Asia 2016**

Mo et al.. *PartNet*. **CVPR 2019**

Dai et al.. *ScanNet*. **CVPR 2017**

Armeni et al.. *S3DIS (Stanford Large-Scale 3D Indoor Spaces)*. **CVPR 2016**

Chang et al.. *Matterport3D*. **3DV 2017**

Geiger et al.. *KITTI*. **CVPR 2012**

Behley et al.. *SemanticKITTI*. **ICCV 2019**

Caesar et al.. *nuScenes*. **CVPR 2020**

Sun et al.. *Waymo Open Dataset*. **CVPR 2020**

Chang et al.. *Argoverse*. **CVPR 2019**

Figures adapted from the original papers and the RRC Summer School source decks; used here for educational purposes.